# 강화학습 Project 레포트

로빛 예비단원 주호진

## 1) 과제

| 구성 | 내용 | 권장 분량 |
|---|---|---|
| 문제·환경 | 목표, 관측·행동·보상·종료 | 1~2쪽 |
| 알고리즘·코드 | DQN과 PPO 비교, 핵심 구현 추적 | 2~3쪽 |
| 실험 설계 | 가설, 변경·고정 변수, 평가 조건 | 1쪽 |
| 결과·고찰 | 그래프, 영상 근거, 해석과 한계 | 2~3쪽 |
| 결론 | 확인된 내용과 후속 실험 | 1쪽 이내 |

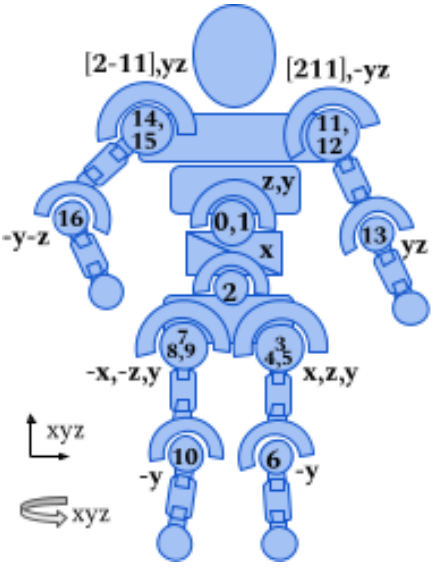
| 구분 | 필수 제출 내용 |
|---|---|
| 보고서 PDF | 본문 6~10쪽, 환경·PPO·코드 분석, 비교 실험 및 고찰 |
| 코드·설정 | GitHub 또는 코드 압축본, README, 패키지 버전<br>기본·변경 설정과 실행 명령, 변경 내역 |
| 실험 결과 | 기본·변경 조건의 로그·그래프·평가 결과<br>모델 checkpoint, 정규화 사용 시 해당 통계 |
| 영상·출처 | 조건별 대표 실행 MP4, 실패 사례 분석<br>참고 예제·문서·AI 사용 범위 명시 |

## 2) 문제·환경

문제: 자세를 유지하면서 최대한 빠르게 전진(+사람처럼 걷기)

환경[1]: Humanoid-v5, 시뮬레이터: MuJoCo

1







| 숫자 | 행동 | 최소 제어 | 컨트롤 맥스 | 이름 (해당 XML 파일에서) | 관절 | 유형(단위) |
|---|---|---|---|---|---|---|
| 0 | 복부의 y좌표에서 힌지에 가해지는 토크 | -0.4 | 0.4 | 복부_y | 들퍼귀 | 토크 (N·m) |
| 1 | 복부의 z 좌표에서 힌지에 가해지는 토크 | -0.4 | 0.4 | 복부_z | 들퍼귀 | 토크 (N·m) |
| 2 | 복부의 x좌표에서 힌지에 가해지는 토크 | -0.4 | 0.4 | 복부_x | 들퍼귀 | 토크 (N·m) |
| 3 | 몸통/복부와 오른쪽 엉덩이 사이의 회전축에 가해지는 토크(x좌표) | -0.4 | 0.4 | 오른쪽 엉덩이 x (오른쪽 허벅지) | 들퍼귀 | 토크 (N·m) |
| 4 | 몸통/복부와 오른쪽 엉덩이 사이의 회전축에 가해지는 토크(z 좌표) | -0.4 | 0.4 | 오른쪽 엉덩이 z (오른쪽 허벅지) | 들퍼귀 | 토크 (N·m) |
| 5 | 몸통/복부와 오른쪽 엉덩이 사이의 회전축에 가해지는 토크(y좌표) | -0.4 | 0.4 | 오른쪽 엉덩이 (오른쪽 허벅지) | 들퍼귀 | 토크 (N·m) |
| 6 | 오른쪽 엉덩이/허벅지와 오른쪽 정강이 사이의 로터에 가해지는 토크 | -0.4 | 0.4 | 오른쪽 무릎 | 들퍼귀 | 토크 (N·m) |
| 7 | 몸통/복부와 왼쪽 엉덩이 사이의 회전축에 가해지는 토크(x좌표) | -0.4 | 0.4 | 왼쪽 엉덩이 x (왼쪽 허벅지) | 들퍼귀 | 토크 (N·m) |
| 8 | 몸통/복부와 왼쪽 엉덩이 사이의 회전축에 가해지는 토크(z 좌표) | -0.4 | 0.4 | 왼쪽 엉덩이 z (왼쪽 허벅지) | 들퍼귀 | 토크 (N·m) |
| 9 | 몸통/복부와 왼쪽 엉덩이 사이의 회전축에 가해지는 토크(y좌표) | -0.4 | 0.4 | 왼쪽 엉덩이 (왼쪽 허벅지) | 들퍼귀 | 토크 (N·m) |
| 10 | 왼쪽 엉덩이/허벅지와 왼쪽 정강이 사이의 로터에 가해지는 토크 | -0.4 | 0.4 | 왼쪽 무릎 | 들퍼귀 | 토크 (N·m) |
| 11 | 몸통과 오른쪽 위팔 사이의 회전자에 가해지는 토크(좌표 -1) | -0.4 | 0.4 | 오른쪽 어깨1 | 들퍼귀 | 토크 (N·m) |
| 12 | 몸통과 오른쪽 위팔 사이의 회전자에 가해지는 토크(좌표 -2) | -0.4 | 0.4 | 오른쪽 어깨2 | 들퍼귀 | 토크 (N·m) |
| 13 | 오른쪽 위팔과 오른쪽 아래팔 사이의 로터에 가해지는 토크 | -0.4 | 0.4 | 오른쪽 팔꿈치 | 들퍼귀 | 토크 (N·m) |
| 14 | 몸통과 왼쪽 위팔(좌표 -1) 사이의 회전자에 가해지는 토크 | -0.4 | 0.4 | 왼쪽 어깨1 | 들퍼귀 | 토크 (N·m) |
| 15 | 몸통과 왼쪽 위팔 사이의 회전자에 가해지는 토크(좌표 -2) | -0.4 | 0.4 | 왼쪽 어깨2 | 들퍼귀 | 토크 (N·m) |
| 16 | 왼쪽 위팔과 왼쪽 아래팔 사이의 로터에 가해지는 토크 | -0.4 | 0.4 | 왼쪽 팔꿈치 | 들퍼귀 | 토크 (N·m) |

**관측, 행동**

17 개의 모터를 가지고 있다.

관측 공간은

위치(22 가지),

속도(23 가지),

질량 중심을 기준으로 한 강체 부품의 질량 및 관성(13*10, 총 130 가지),

몸통, 허리, 골반, 허벅지(좌, 우), 종아리(좌, 우), 발(좌, 우), 위팔(좌, 우), 아래팔(좌, 우)

질량중심속도(COM 속도, 78 가지),

엑추에이터 힘(위 사진 참고, 17 가지),

질량중심기반 외력(78 가지)이 있고 총 348 가지이다. 관측함수는 이 348 가지의 정보를 뱉는다.

Humanoid-v5 의 action space 는 17 개의 값을 가진다.(모터 제어)

휴머노이드의 링크 길이나 모터 최대 토크, 기어비 등의 정보는 xml[2] 파일에 정의되어있다. 해당 값을 사용해 실제와 같은(비슷한) 환경을 구성하는 시뮬레이션을 만든다.

**종료 조건**

Humanoid-v5 의 몸통 높이(z 좌표)가 지정된 범위를 벗어났을 때(기본 값 z 값 [1, 2]) unhealthy 하다고 판단하고, terminated 시킨다.

---

2

넘어지거나, 이상한 보행(무릎을 굽힌다던지, 모터 계산이 잘못 들어가 로봇이 튀어(?)오른다던지 등)

1000 step 달성 시 Truncation 처리한다.

**초기화**

계속되는 학습에서 동일한 시드에서 학습할 경우 해당 시드에만 특화되는 과적합이 일어날 수 있기 때문에 랜덤한 시드값을 사용하는 것이 좋다. (reset 시 기본 설정 random)

### 3) 코드

코드 레퍼런스

https://huggingface.co/hwihwalab/neuromotion-humanoid-v5-ppo/blob/9539b5da3fdc89520d1d6ba02c9c2646b45b7c80/mujoko_humanoid.py

```
lr = 0.0003
ctrl_cost_weight = 1.5
forward_reward_weight = 0.6
policy_kwargs = dict(net_arch=dict(pi=[256, 256], vf=[256, 256])) # actor, critic
```

위 코드 레퍼런스를 기반으로 변경 변수를 위로 빼주었다.

```
# 고속 학습용 환경 (render_mode=None)
env = make_humanoid_env(render_mode=None)

model = PPO(
    policy=policy,
    env=env,
    learning_rate=lr,
    n_steps=2048,
    batch_size=batch_size,
    n_epochs=epoch,
    gamma=0.99,
    gae_lambda=0.95,
    clip_range=0.2,
    verbose=1,
    policy_kwargs=policy_kwargs,
)
```

```
try:
    model.learn(total_timesteps=total_timesteps, callback=[plot_cb, visual_cb])
    print(f"\n학습 완료! 모델을 저장합니다 -> {model_save_path}")
    model.save(model_save_path)
```

학습을 하기 위한 설정을 기입해주고, 설정한 step 까지 학습완료 후 model 을 저장한다.

```
env = gym.make("Humanoid-v5", render_mode="human
model = PPO.load(model_save_path, env=env)
```

평가 시에 화면을 보기 위해 render mode 를 human 으로 지정해주고, 저장된 model 을 불러와서 사용한다.

```
eval_forward_velocities = []
eval_ctrl_costs = []
eval_survival_steps = []
```

```
for ep in range(n_episodes):
```

확인할 지표를 위한 list 를 선언하여 준 뒤, 평가를 진행한다(20 회).

### 4) 실험 설계

실험 환경

OS

ubuntu 24.04

Package

Python 3.12.3

stable_baselines3 2.9.2

gymnasium 1.4.0

mujoco 3.17.0

Equipment(cpu 사용)

cpu: 12th Gen Intel(R) Core(TM) i7-1260P

RAM: 32GB

변수는 PPT 에 제시된 변경 변수 4 개만 설정한 뒤 초기 실험을 진행한다.

학습률은 레퍼런스 코드에 있는 부분을 그대로 사용, 나머지 3 개의 변수는 gemini 를 통해 기본값을 설정해달라고 하여 해당 값을 집어 넣었다.

설정 step 100,000

### 5) 결과·고찰

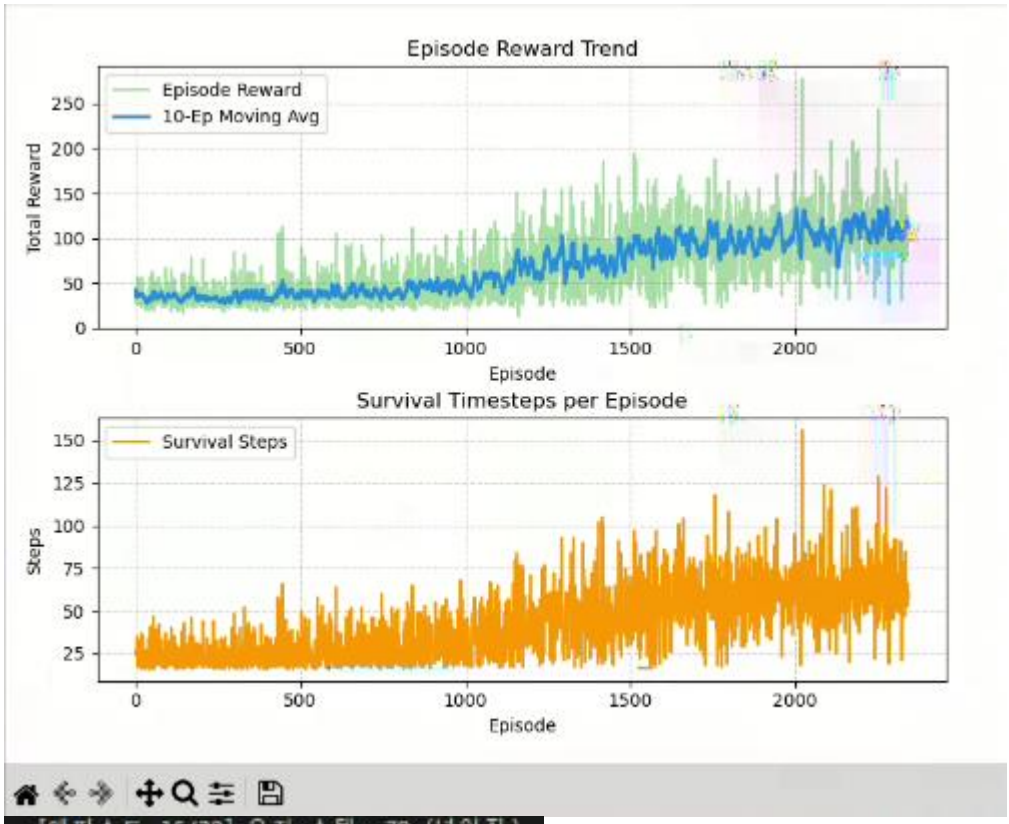
초기 실험

소요 시간: 약 8 분

영상

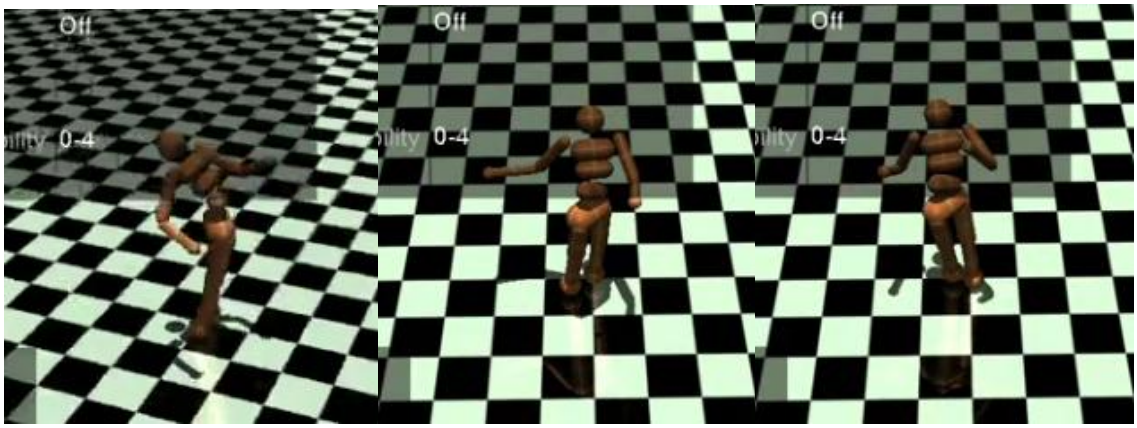
https://drive.google.com/file/d/19KZlR1-YhFc2DywVGzL-gIoUSkJMOu74/view?usp=sharing

```
lr = 0.0003
ctrl_cost_weight = 1.5
forward_reward_weight = 0.6
policy_kwargs = dict(net_arch=dict(pi=[256, 256], vf=[256, 256])) # actor, critic
```




```
[에피소드 16/20] 유지 스텝: 79 (넘어짐)
[에피소드 17/20] 유지 스텝: 73 (넘어짐)
[에피소드 18/20] 유지 스텝: 72 (넘어짐)
[에피소드 19/20] 유지 스텝: 74 (넘어짐)
[에피소드 20/20] 유지 스텝: 68 (넘어짐)

=== 최종 20회 평가 결과 평균 ===
평균 에피소드 유지 시간(스텝): 81.0
평균 전진 속도: 0.3446 ± 0.0921
평균 제어 입력: 0.0883 ± 0.0024
```




초기 실험 관찰 결과 평가 episode 에서 전부 허리가 먼저 튕겨나가면서 넘어지게 됨.

Reward 와 episode 별 step 이 전반적으로 우상향 하는 것을 보았을 때 학습률은 조정할 필요가 없다고 판단하였음.

허리가 앞으로 나아가는 것이 전진 보상 가중치에 대한 reward 를 얻기 위해 허리가 튕겨나가지는 것은 아닐까 생각하여 전진 보상 가중치만 수정하여 변경 실험을 진행해보고자 함.

전진 보상 가중치가 줄어든다면 -> 전진할 때 reward 가 줄어들기 때문에 허리가 앞으로 나아가지는 현상이 줄어들 것으로 예상, 그리고 허리가 튕겨지는 현상이 줄어듦으로 인해 평균 전진 속도도 줄어들것이라고 예상하고 변경 실험을 진행함.

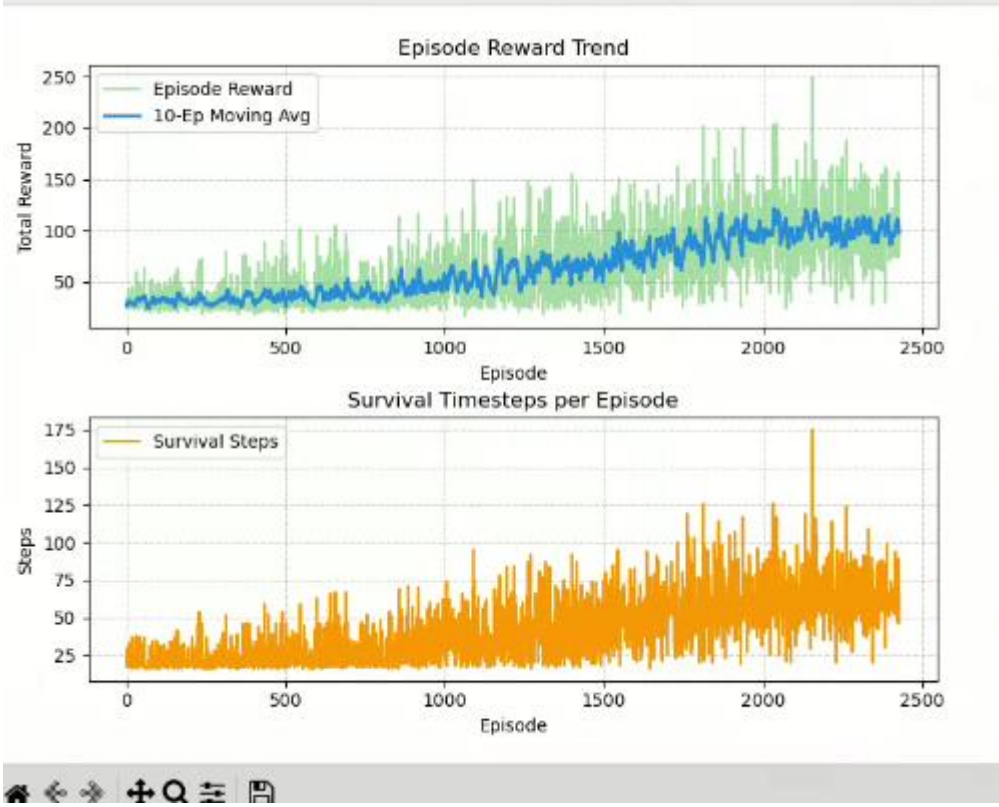
변경 실험

소요 시간: 약 9 분 30 초

영상

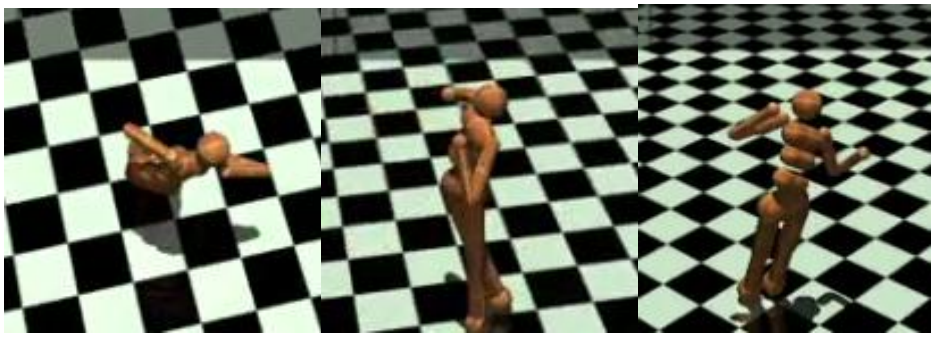
https://drive.google.com/file/d/1dO3yNm4CefIF3CLwWLaZnyoWKZGDAI4h/view?usp=sharing

```
lr = 0.0003
ctrl_cost_weight = 1.5
forward_reward_weight = 0.6
policy_kwargs = dict(net_arch=dict(pi=[256, 256], vf=[256, 256]))
```




```
[에피소드 17/20] 유지 스텝: 57 (넘어짐)
[에피소드 18/20] 유지 스텝: 56 (넘어짐)
[에피소드 19/20] 유지 스텝: 58 (넘어짐)
[에피소드 20/20] 유지 스텝: 62 (넘어짐)

=== 최종 20회 평가 결과 평균 ===
평균 에피소드 유지 시간(스텝): 56.2
평균 전진 속도: 0.4485 ± 0.0455
평균 제어 입력: 0.0943 ± 0.0026
==================================
```



### 6) 결론

전진 보상 가중치를 1.0 에서 0.7 로 줄여 실험을 진행했을 때, 오히려 기존 조건보다 기능이 떨어지는 것을 확인할 수 있었음.

예측한 대로 허리가 튕겨져 나가는 현상이 시뮬레이션 상으로 줄어든 것은 확인되나, 오히려 허리가 꺾이지 않음으로 인해서 몸통이 허리보다 앞으로 나아가면서 상체 전체가 넘어지는 현상이 발생하게 됐음. 이에 따라 허리가 먼저 튕겨져 나갈 때보다 쓰러지는 속도가 빨라서인지, 평균 전진 속도도 기존 실험에 비해 높은 값이 뜨게 되었음.

휴머노이드 보행에 세팅하는 step 은 적어도 100 만 단위 이상이라는 자료를 봤을 때 10 만이라는 step 세팅은 실험 결과를 확인하기에는 적절하지 않은 step 이기도 하다.

추가 실험을 진행하기 위해서는 일단 step 설정을 기존 10 만에서 100 만정도는 올려서 테스트를 진행해야 유의미한 결과값을 확인할 수 있을 것 같다.

---

### 7) DQN PPO 비교

| 구분 | CartPole DQN | Humanoid PPO |
|---|---|---|
| 행동 | 좌우 2개 중 선택 | 17개 제어 입력 동시 결정 |
| 정책 표현 | 행동별 Q 값으로 행동 선택 | 연속 행동 분포를 이용한 정책 |
| 학습 대상 | Q-Network | 정책과 상태 가치 함수 |

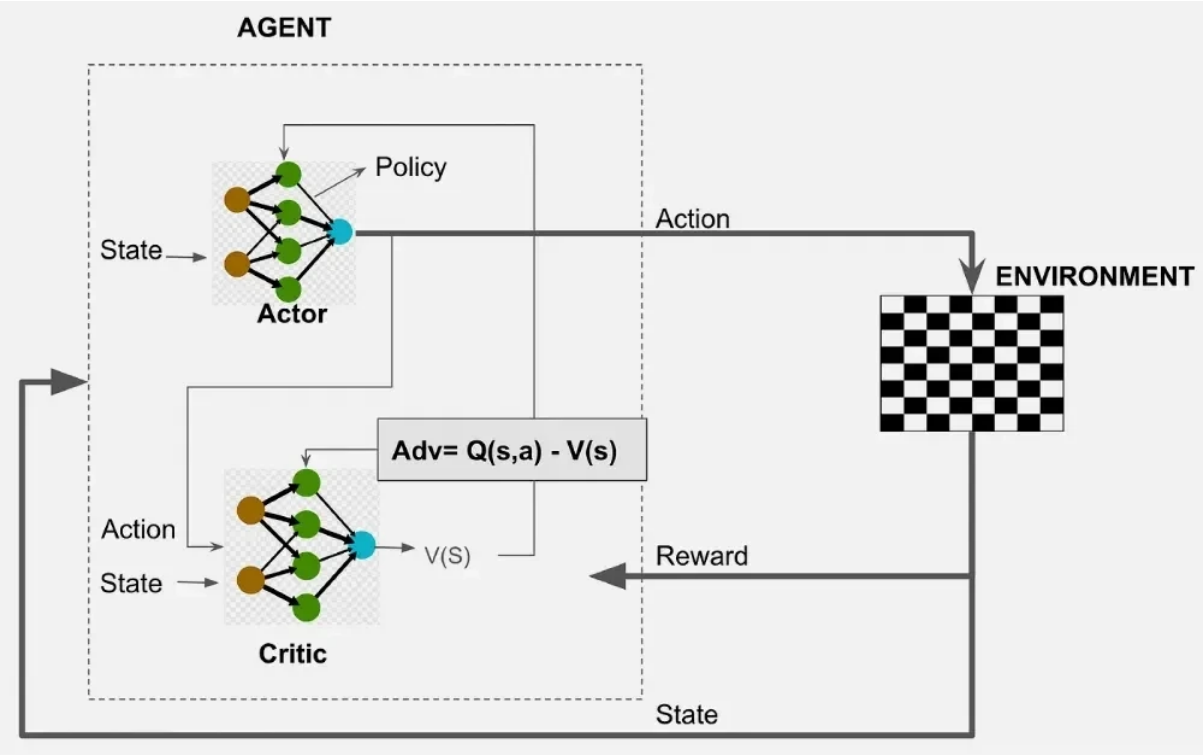
DQN 은 이산적인 환경에서는 잘 작동하나, 행동 공간이 고차원적이거나 연속적인 환경에서는

Qmax 를 구할 때 계산해야 할 연산이 기하급수적으로 늘어나기 때문에 매우 비효율적이게 된다.[3]

## 8) PPO 분석(+코드)

### Actor-Critic

Actor-critic 은 정책을 학습하는 네트워크와 행동의 가치를 평가하는 네트워크를 동시에 학습하는 구조를 가진다.[4]




Actor 는 현재 상태를 입력받아 취할 수 있는 행동의 확률 분포를 출력한다. 현재 상태에서 어떤 행동을 해야 하는지 결정한다.

Critic 은 Actor 의 판단에 대한 점수를 매긴다.

stable_baselines3/common/policies.py class ActorCriticPolicy

3

4

```
644             # Preprocess the observation if needed
645             features = self.extract_features(obs)
646             if self.share_features_extractor:
647                 latent_pi, latent_vf = self.mlp_extractor(features)
648             else:
649                 pi_features, vf_features = features
650                 latent_pi = self.mlp_extractor.forward_actor(pi_features)
651                 latent_vf = self.mlp_extractor.forward_critic(vf_features)
652             # Evaluate the values for the given observations
653             values = self.value_net(latent_vf)
654             distribution = self._get_action_dist_from_latent(latent_pi)
655             actions = distribution.get_actions(deterministic=deterministic)
656             log_prob = distribution.log_prob(actions)
657             actions = actions.reshape((-1, *self.action_space.shape))  # type: ignore[misc]
658             return actions, values, log_prob
659
```

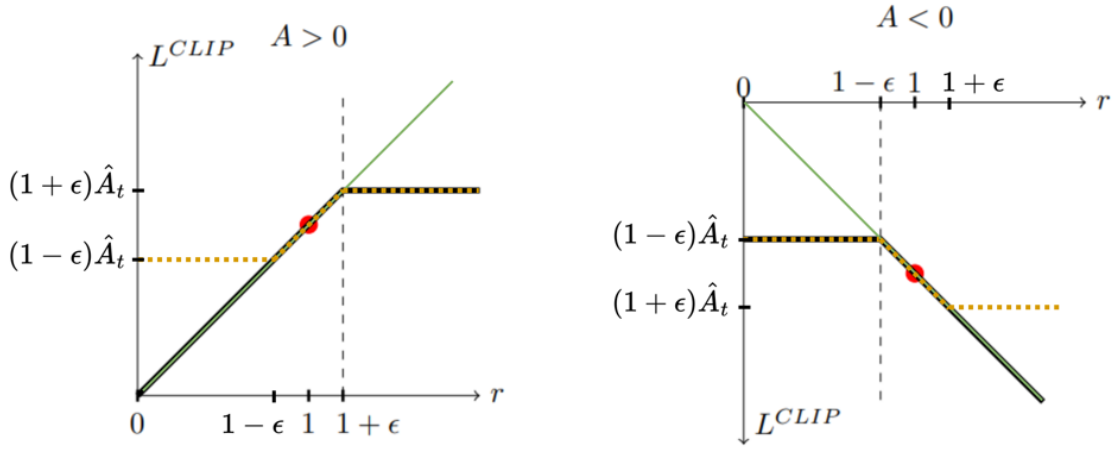
Pi = policy(Actor), vf = value function(Critic), log_prob: Actor 가 선택한 action 이 작동할만한(합당한) 행동이었는지를 나타내는 확률에 로그를 씌운 것

이때, 정책을 학습하는 네트워크와 행동의 가치를 평가하는 네트워크를 동시에 학습하는 구조를 가지면 DQN 처럼 정답이 계속해서 바뀌면 발산하거나 진동할 수 있기 때문에 다음과 같은 안전 장치를 Actor-Critic 구조에서 보완한다. (PPO)

## PPO clipping

Clipping 은 actor 의 업데이트 폭을 (1 – epsilon, 1 + epsilon)으로 제한하여 특정 action 의 확률이 급격하게 증가하거나 감소하는 것을 억제하는 역할을 한다.

$$L^{\text{CLIP}}(\theta) = \min\left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)$$




stable_baselines3/ppo/ppo.py class PPO line 224

```
# clipped surrogate loss
policy_loss_1 = advantages * ratio
policy_loss_2 = advantages * th.clamp(ratio, 1 - clip_range, 1 + clip_range)
policy_loss = -th.min(policy_loss_1, policy_loss_2).mean()
```

## Rollout Buffer

롤아웃 버퍼는 최신 경험 데이터를 임시로 저장하는 메모리 공간이다.[5]

PPO 는 On-Policy Rollout Buffer 을 사용하여 소수의 최신 경험 데이터를 사용한다. 해당 경험 데이터를 사용한 후에는 폐기 후, 다시 새로운 데이터를 수집한다.

DQN 은 Off-Policy Rollout Buffer 을 사용한다. On policy 와 다르게 최신 데이터를 사용하는 것이 아닌 메모리 공간 내에 데이터들을 무작위로 사용한다는 점에서 차이가 있다.

stable_baselines3/common/buffers.py class ReplayBuffer

```
class RolloutBuffer(BaseBuffer):
```

```
def add(
    self,
    obs: np.ndarray,
    action: np.ndarray,
    reward: np.ndarray,
    episode_start: np.ndarray,
    value: th.Tensor,
    log_prob: th.Tensor,
) -> None:
```

## Return·Advantage

롤아웃 버퍼에 저장한 데이터를 어디에 사용하느냐 하면 Return 과 advantage 을 계산할 때 사용하게 된다.

Return 은 앞에서도 계속 다뤘던 expect return 값이고, advantage 는 특정 행동을 했을 때 그 행동이 기댓값보다 얼마만큼 이익(또는 손해)이 있었는가를 나타내는 값이다. policy 를 개선하는 손실 함수의 요소가 된다.

---

5

stable_baselines3/common/buffers.py class ReplayBuffer line 403

```
def compute_returns_and_advantage(self, last_values: th.Tensor, dones: np.ndarray) -> None:

# Convert to numpy
last_values = last_values.clone().cpu().numpy().flatten()  # type: ignore[assignment]

last_gae_lam = 0
for step in reversed(range(self.buffer_size)):
    if step == self.buffer_size - 1:
        next_non_terminal = 1.0 - dones.astype(np.float32)
        next_values = last_values
    else:
        next_non_terminal = 1.0 - self.episode_starts[step + 1]
        next_values = self.values[step + 1]
    delta = self.rewards[step] + self.gamma * next_values * next_non_terminal - self.values[step]
    last_gae_lam = delta + self.gamma * self.gae_lambda * next_non_terminal * last_gae_lam
    self.advantages[step] = last_gae_lam
# TD(lambda) estimator, see Github PR #375 or "Telescoping in TD(lambda)"
# in David Silver Lecture 4: https://www.youtube.com/watch?v=PnHCvfgC_ZA
self.returns = self.advantages + self.values
```

## 손실 함수

PPO 에서 손실함수는 정책 손실(policy loss), 가치 손실(value loss), 엔트로피 보너스(entropy bonus) 이 세가지가 조합된 형태로 구성된다. 정책 손실은 위에서 설명한 advantage 가 반영된 값이고(clipping 적용), 가치 손실은 critic 에서 예측한 가치와 버퍼에서 계산된 return 간의 오차를 반영한 값이다. 엔트로피 보너스는 학습 초기에 특정 행동만으로 학습되는 것을 막도록 행동에 무작위성을 부여하는 것이다. (입실론 그리디의 입실론의 경우)

stable_baselines3/ppo/ppo.py class PPO line 221 ~

```
# ratio between old and new policy, should be one at the first iteration
ratio = th.exp(log_prob - rollout_data.old_log_prob)

# clipped surrogate loss
policy_loss_1 = advantages * ratio
policy_loss_2 = advantages * th.clamp(ratio, 1 - clip_range, 1 + clip_range)
policy_loss = -th.min(policy_loss_1, policy_loss_2).mean()

# Value loss using the TD(gae_lambda) target
value_loss = F.mse_loss(rollout_data.returns, values_pred)
value_losses.append(value_loss.item())
```

```
# Entropy loss favor exploration
if entropy is None:
    # Approximate entropy when no analytical form
    entropy_loss = -th.mean(-log_prob)
else:
    entropy_loss = -th.mean(entropy)

entropy_losses.append(entropy_loss.item())

loss = policy_loss + self.ent_coef * entropy_loss + self.vf_coef * value_loss
```

### 9) 참고문헌


[1] Gymnasium Documentation [humanoid], https://gymnasium.farama.org/environments/mujoco/humanoid/.

[2] Github google-deepmind [humanoid.xml], https://github.com/google-deepmind/mujoco/blob/main/model/humanoid/humanoid.xml.

[3] Timothy P. Lillicrap et al., "CONTINUOUS CONTROL WITH DEEP REINFORCEMENT LEARNING", arxiv, https://arxiv.org/pdf/1509.02971, 2016, pp 1-2.

[4] tistory [강화학습 알고리즘: (5) PPO], https://ai-com.tistory.com/entry/RL-%EA%B0%95%ED%99%94%ED%95%99%EC%8A%B5-%EC%95%8C%EA%B3%A0%EB%A6%AC%EC%A6%98-5-PPO.

[5] agilerl [On-Policy Rollout Buffer]

, https://docs.agilerl.com/en/latest/api/components/rollout_buffer.html.